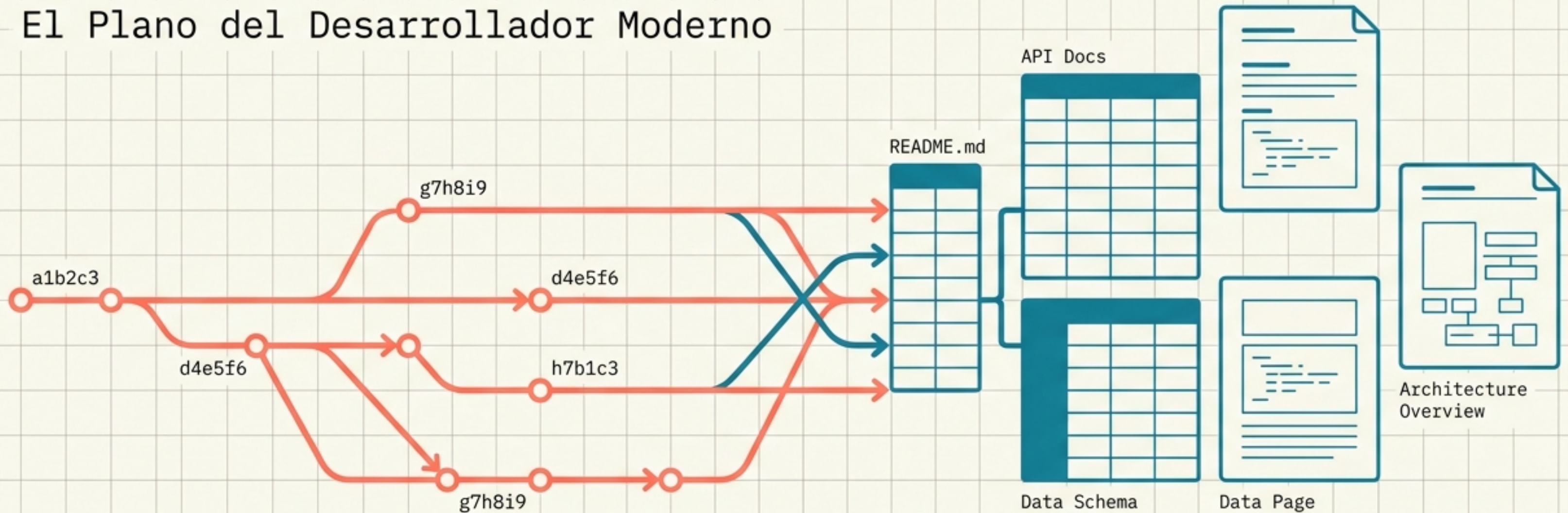


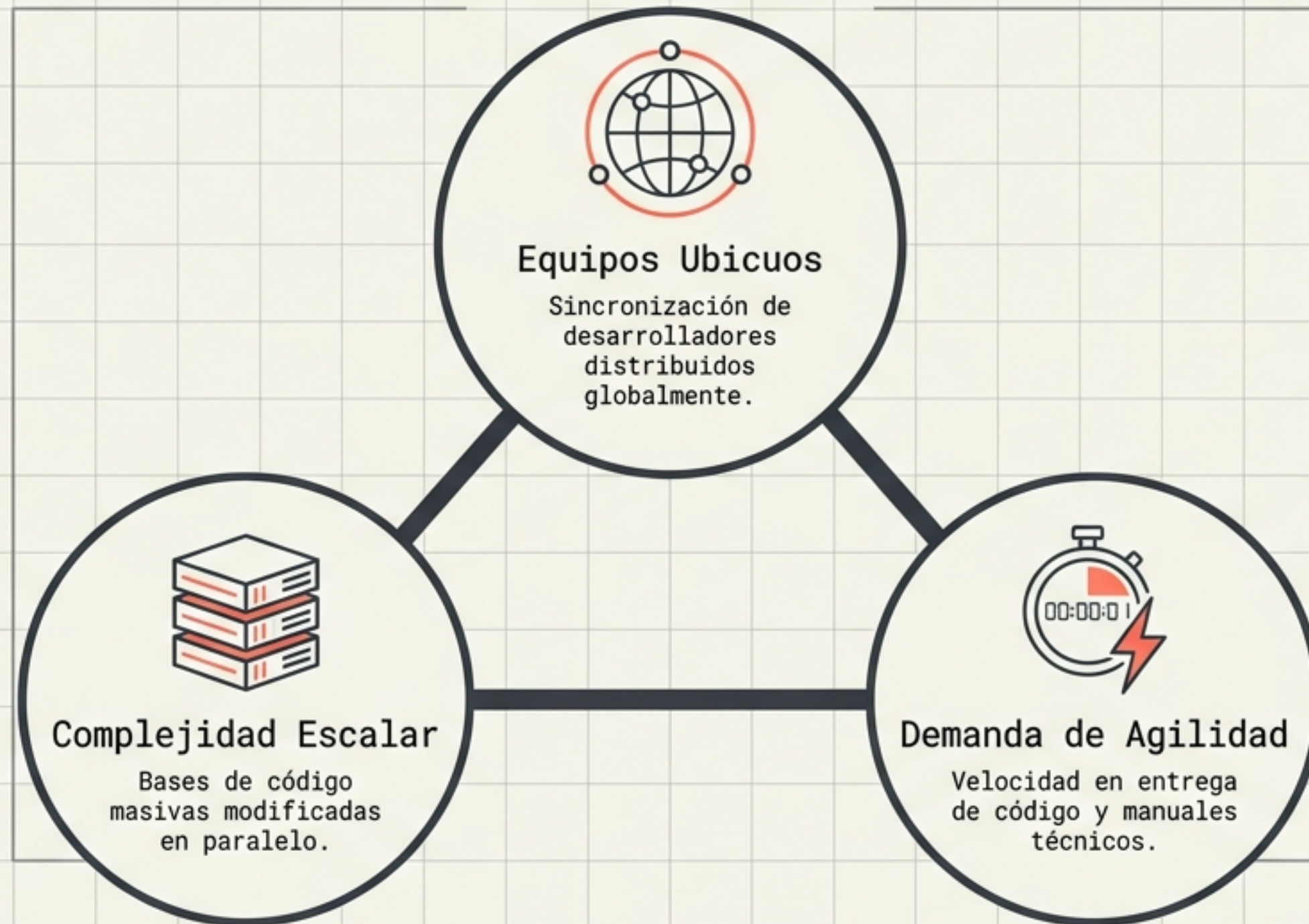
Control de Versiones y Documentación

El Plano del Desarrollador Moderno



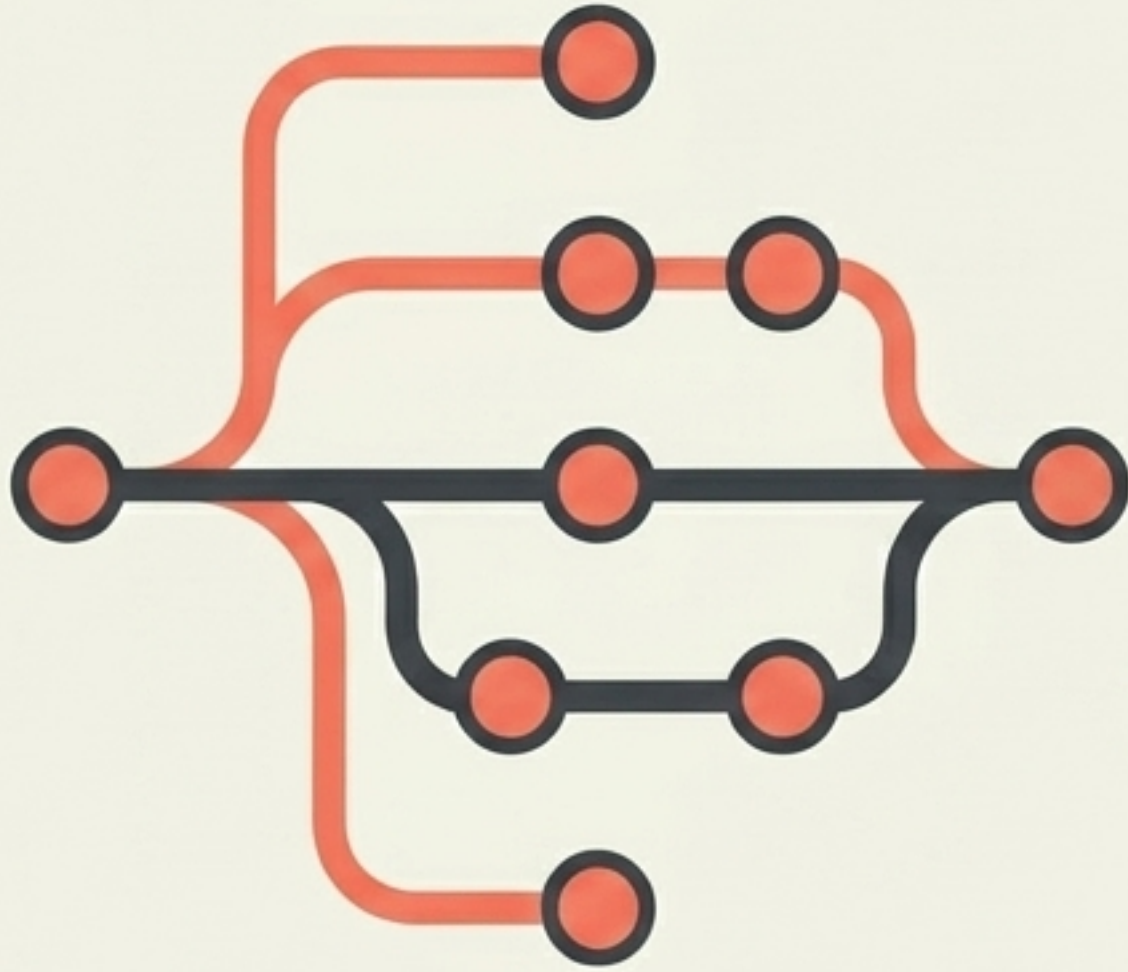
Fuente: Tema 18. Entornos de desarrollo

El Reto del Desarrollo Moderno



Sin las herramientas adecuadas, el desarrollo colaborativo degenera en caos.

Los Dos Pilares del Orden



Sistema de Control de Versiones (SCV)

Gestiona el código, resuelve conflictos y sincroniza al equipo.

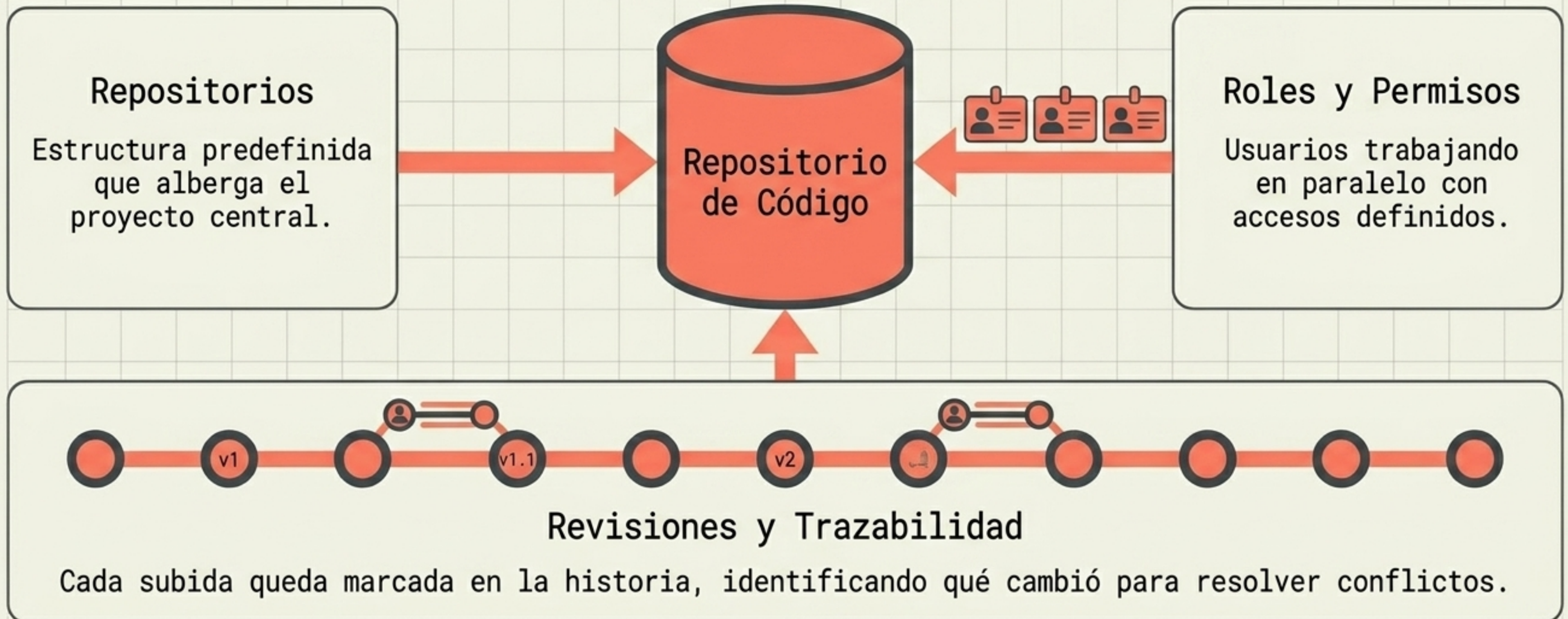


Ecosistema de Documentación

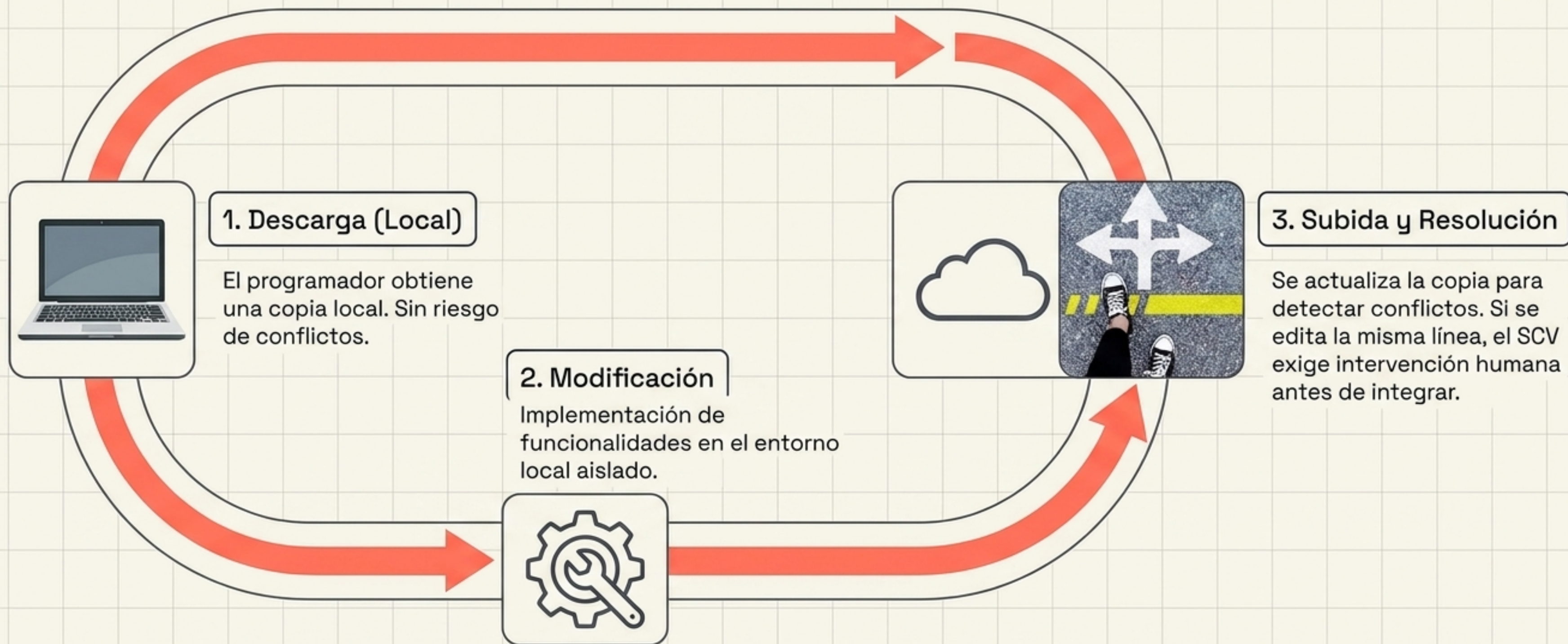
Preserva la lógica, guía a los usuarios y asegura el mantenimiento futuro.

¿Qué es un Sistema de Control de Versiones?

Una infraestructura que permite controlar y revisar periódicamente el código generado por equipos grandes.



El Flujo de Trabajo SCV



Proyecto global, millones de líneas de código. ¿Sirve Dropbox para sincronizar el equipo?

Dropbox



SCV / Git



Objetivo

Almacenamiento general de archivos.

Gestión específica de código fuente.

Edición Simultánea

Sobrescribe archivos o crea duplicados confusos.

Permite múltiples ediciones estructuradas en el mismo archivo.

Resolución de Conflictos

Inexistente.

Unión inteligente; alerta si se modifica la misma línea.

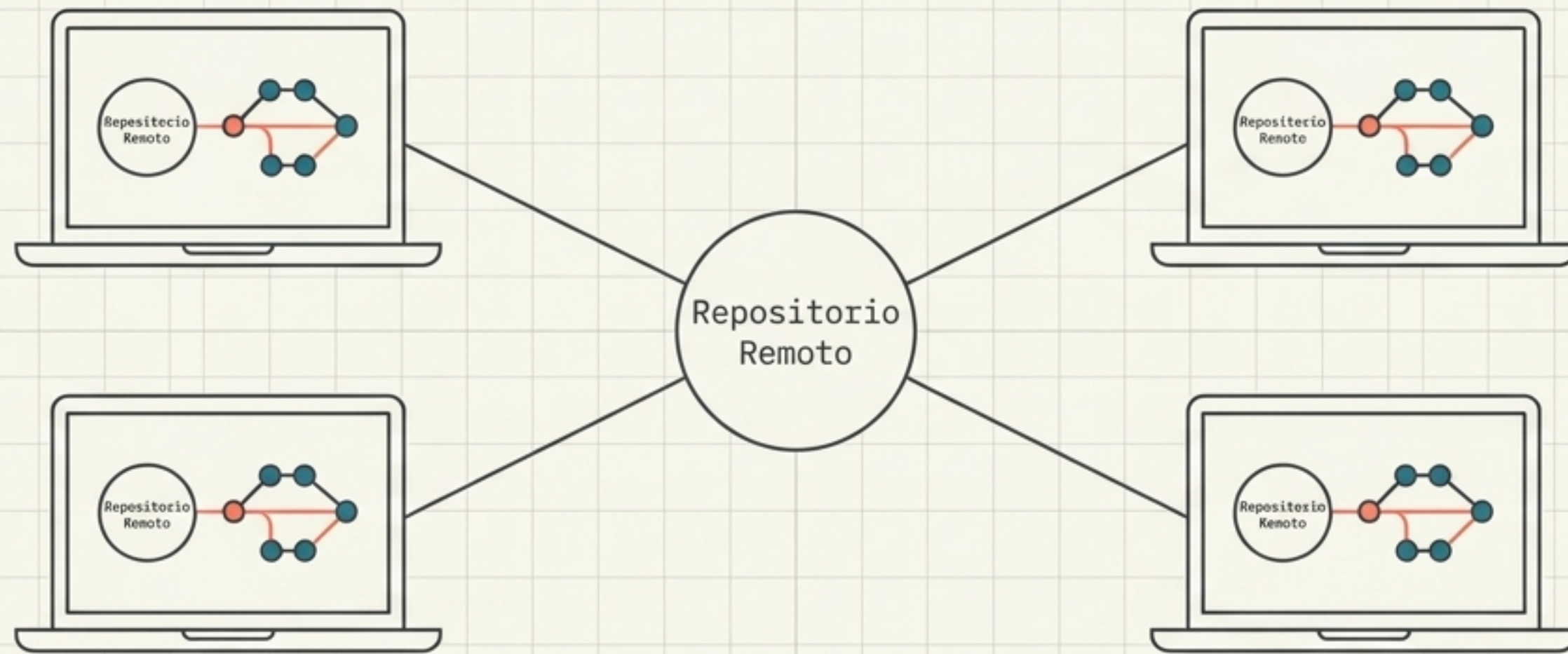
Trazabilidad

Historial básico a nivel de archivo.

Historial detallado línea por línea.

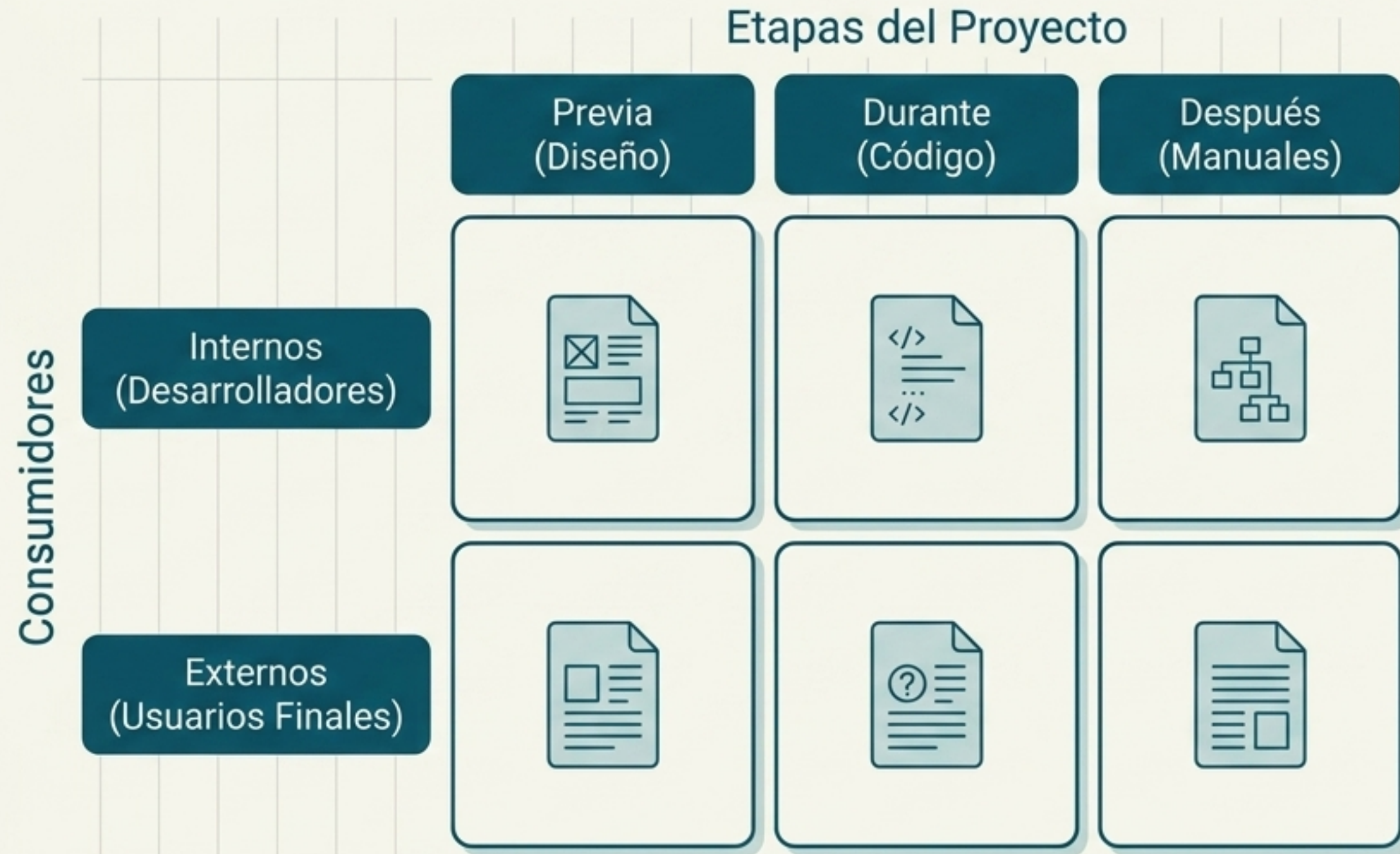
Un Vistazo a Git: El Estándar Distribuido

Git es uno de los sistemas de control de versiones más utilizados en la actualidad debido a su arquitectura distribuida.



Repositorios Locales Completos: A diferencia de los sistemas centralizados, cada usuario que trabaja en el proyecto posee una copia local completa de todo el repositorio, maximizando la velocidad y la seguridad de los datos.

El Ecosistema de la Documentación



La documentación no es un paso final; es un proceso continuo con audiencias radicalmente distintas.

El Primer Nivel: Comentarios en el Código

```
/*  
Este bloque calcula el resultado final  
basado en las entradas y aplica  
una tasa de procesamiento específica.  
*/  
function calcularResultado(entradas) {  
  let resultado = 0; // Inicializa el resultado  
  const tasa = 1.25;  
  for (let i = 0; i < entradas.length; i++) {  
    resultado += entradas[i] * tasa;  
  }  
  return resultado; // Devuelve el valor calculado  
}
```

Comentario
de bloque

Comentario
de línea

Tipología:

Inserción sencilla. Son parte integral del propio código.

El Peligro de la Refactorización:

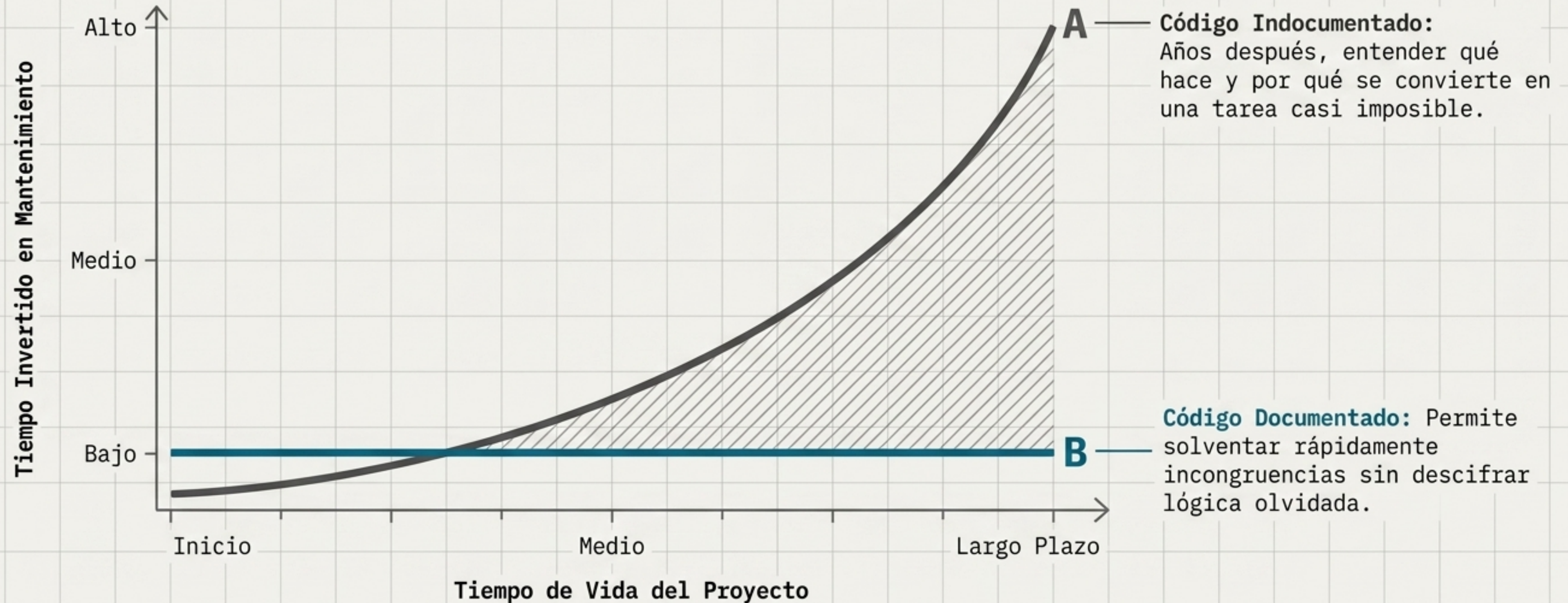
Cualquier modificación en la lógica del código exige la actualización inmediata del comentario.



Un comentario desactualizado es peor que no tener comentarios: confunde al desarrollador simulando un comportamiento falso.

La Ecuación del Mantenimiento

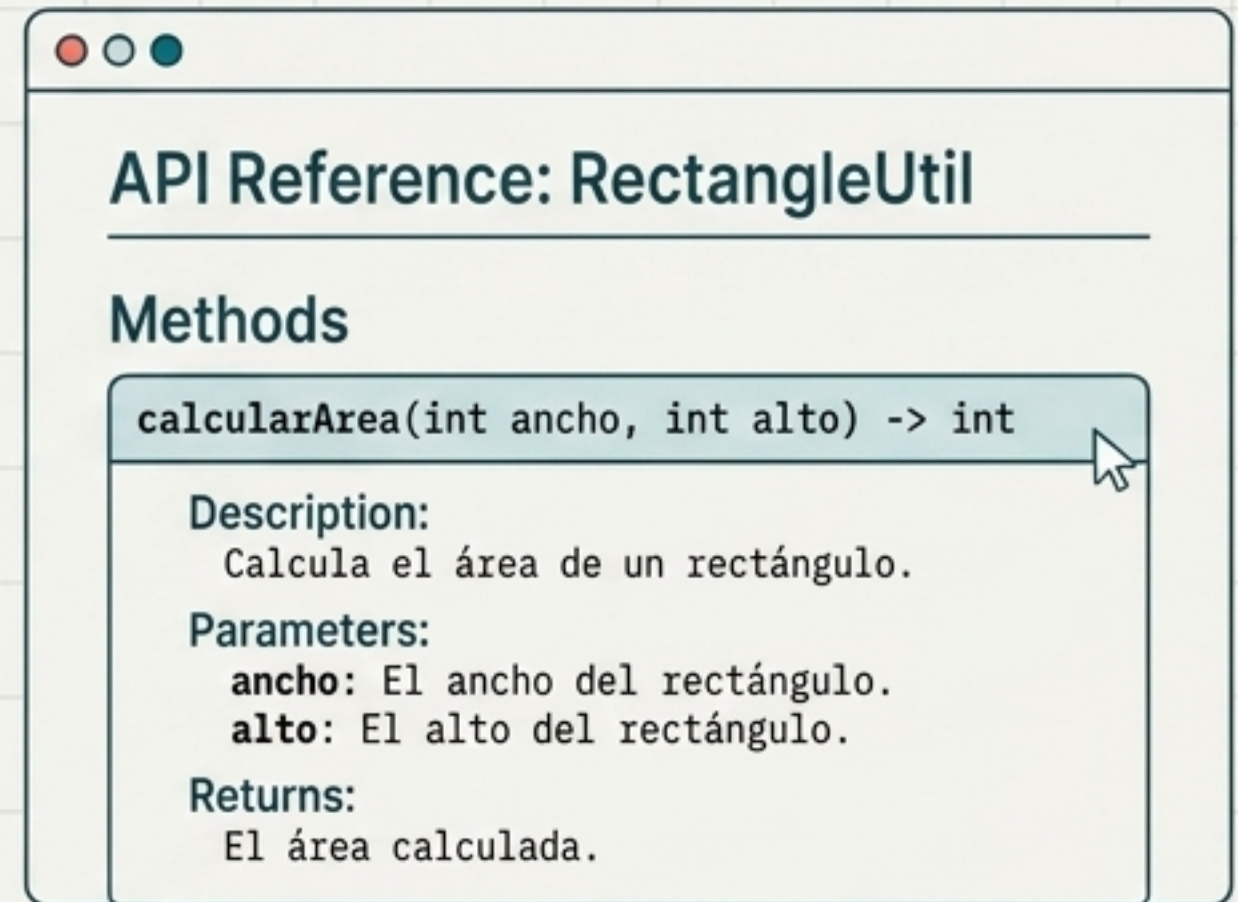
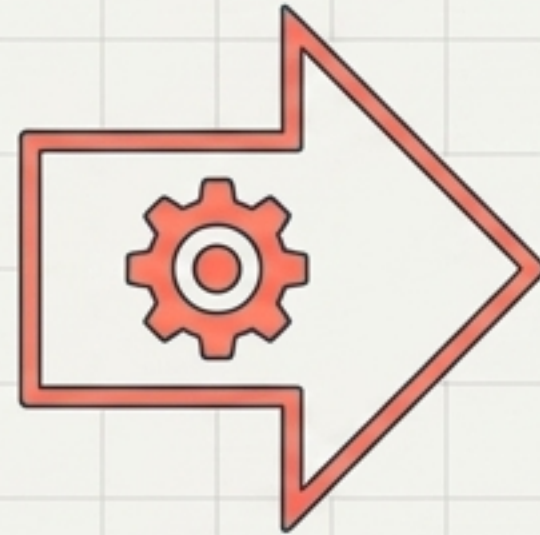
Al principio, comentar el código parece un coste de tiempo.
A largo plazo, es una inversión crítica.



Generación Automática de Documentación

Herramientas integradas o externas que analizan el código y extraen la información.

```
/**
 * Calcula el área de un rectángulo.
 * @param ancho El ancho del rectángulo.
 * @param alto El alto del rectángulo.
 * @return El área calculada.
 */
public int calcularArea(int ancho, int alto) {
    return ancho * alto;
}
```



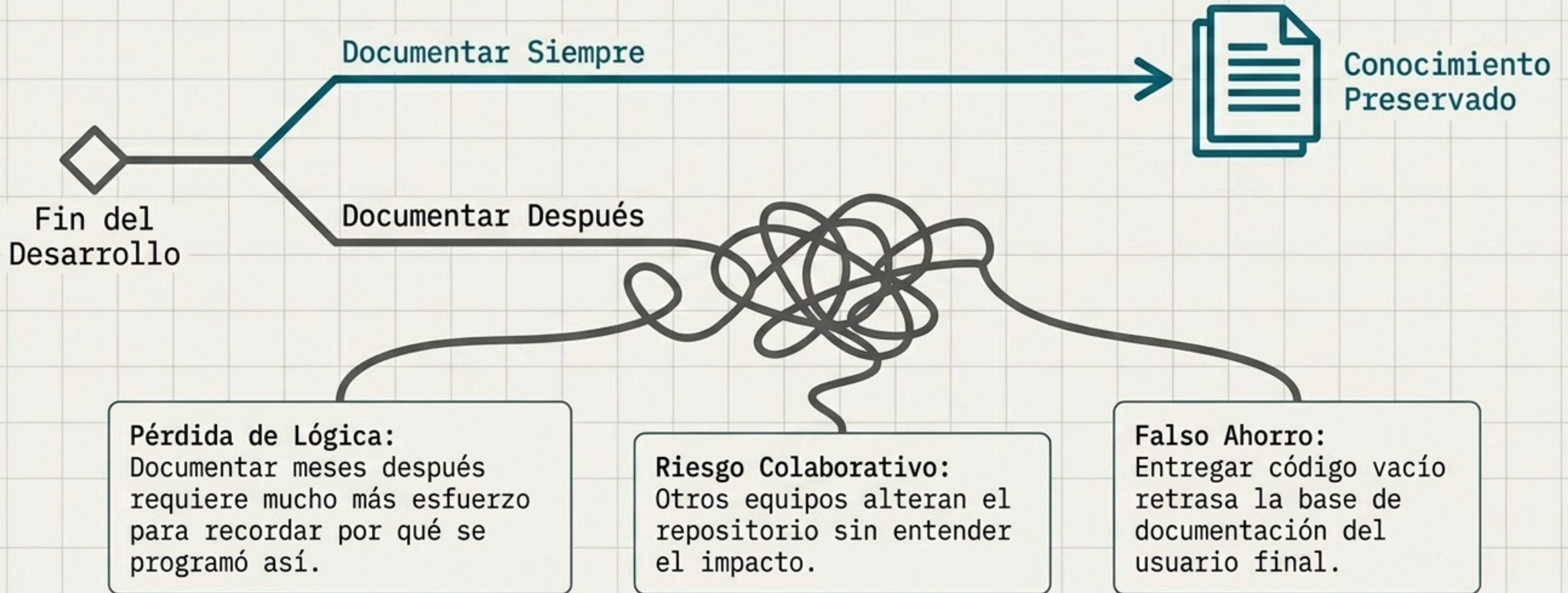
1. Sintaxis de Etiquetas: Se introducen campos determinados directamente en el código fuente.

2. Extracción: La herramienta compila estas etiquetas (Ej: Javadoc).

3. Resultado: Documentación técnica detallada, ideal para APIs (pero no diseñada como manual de usuario final).

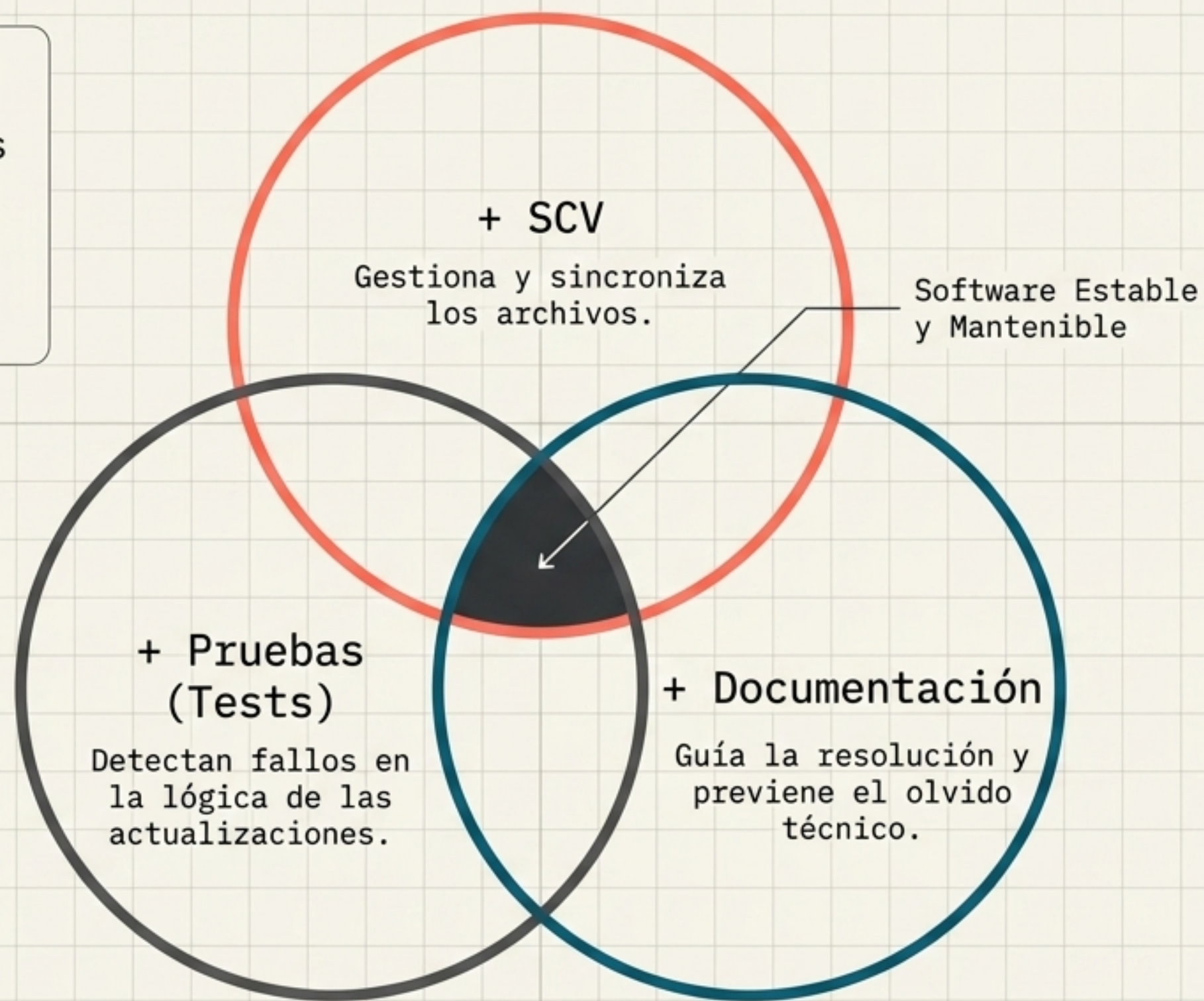
Reality Check: La Falacia de "Documentar Después"

El jefe pide entregar el software ahora y "ya volveremos al código cuando se pueda" para documentarlo.



Síntesis: Resolviendo el Caos (Caso Final)

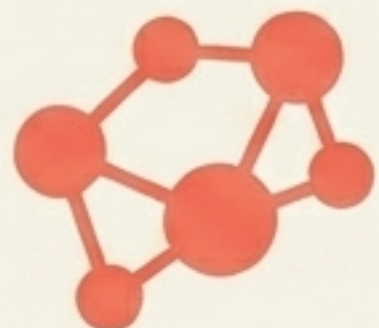
Problema: El cliente reporta errores tras las actualizaciones.
El equipo argumenta: "¡Pero usamos un SCV!"



La Realidad:
Un SCV no evita errores de lógica.

Resolver conflictos incorrectamente al subir puede código puede incluso introducir nuevos bugs si no hay pruebas ni documentación.

Conclusiones Clave



Imprescindible en Equipo

Los SCV son el único método viable para unificar desarrollos complejos sin destruir el progreso ajeno.



Flujo Local -> Remoto

Modifica en aislamiento, actualiza, resuelve conflictos humanos, y sube al repositorio central.



Comentar es Invertir

El primer nivel de documentación ahorra incontables horas de ingeniería inversa en el futuro.



Ecosistema Completo

El control de versiones sin documentación ni pruebas es solo una base de datos de código inestable.

Bibliografía

Price, J. & Korman, H. (1993). How to communicate technical information: a handbook of software and hardware documentation. Redwood City, Calif: Benjamin/Cummings Pub. Co.

Clements, P. (2011). Documenting software architectures: views and beyond. Upper Saddle River, NJ: Addison-Wesley.

Swicegood, T. & Pfalzer, S. (2012). Pragmatic version control using Git. Place of publication not identified: Pragmatic Bookshelf.